

1 Question 1

Part(a) Naive solution:

For each potential shift $-(n-1)$ to $n-1$, iterate over the range and calculate the number of matches. Store the current highest value with associated shift.

Part(b) $\exists$ 26 alphabets and one space. This gives us a total of 27 characters. For each character c , we define 2 binary vectors X_c and Y_c such that:

$$X_c[i] = \begin{cases} 1 & A_1[i] = c \\ 0 & \text{otherwise} \end{cases}$$

$$Y_c[i] = \begin{cases} 1 & A_2[i] = c \\ 0 & \text{otherwise} \end{cases}$$

Note that for each i $\exists$ exactly one of the $X_c[i]$ and $Y_c[i]$ equals as a distinct idx can only be occupied by one unique character. Each of these vectors corresponding to a character allows us to define a polynomial where the values in the vector represent the coefficients. These are represented as:

$$P_c(x) = \sum_{i=0}^{n-1} X_c[i]x^i; Q_c(x) = \sum_{j=0}^{n-1} Y_c[j]x^j.$$

Multiplying the polynomials via FFT gives:

$$R_c(x) = P_c(x) \times Q_c(x) = \sum_{k=0}^{2(n-1)} Z_c[k]x^k,$$

where

$$Z_c(k) = \sum_{i+j=k} X_c[i].Y_c[j].$$

Since $X_c[i]$ and $Y_c[j]$ can only be 1 when the cells in both A_1 and A_2 contain the same characters. We can sum these products to count the number of pairs $(i, j) : i + j = r$ that have the same character.

In order to get the polynomial multiplication to give us the effect of shifting A_2 we must reverse the other polynomial such that

$$Q_{inv}(x) = \sum_j q_j x^{n-1-j}.$$

Now, polynomial multiplication gives

$$\sum_{i+(n-1-j)=k} p_i q_j = \sum_{(j-i)=(n-1-k)} p_i q_j;$$

now define the summation over shift $(j - i) = l$.

For a given shift, the total number of matches is:

$$\sum \text{no. of indices } i: A_1[i] = c \wedge A_2[i + l] = c$$

where l represents the shift. Therefore, add up the coefficient for $R_c(x)$ $\forall c$ and the largest of these sums over possible shifts k is the one with maximum characters aligned.

2 Question 2

- **Part (a):** The solution below does not take into account the correction (+) made during the exam. Regardless, almost all the steps remain the same.

To substitute: $S(n) = T(n) - c.f(n) = T(n) - c.n^p$. Substituting this in the given equation for $S(n)$ gives:

$$T(n) - c.n^p = a.T(n/b) - a.c(n/b)^p.$$

Substituting the recursion for $T(n)$ in L.H.S. above gives:

$$[a.T(n/b) - n^p] - c.n^p = a.T(n/b) - a.c(n/b)^p.$$

Cancelling terms on both sides gives:

$$-n^p - c.n^p = -a.c(n/p)^p \implies 1 + c = acb^{-p}$$

$$\implies ab^{-p}c - c = 1 \implies c = \frac{1}{ab^{-p} - 1} = \frac{b^p}{a - b^p}$$

The denominator must be non-zero: $ab^{-p} \neq 1 \implies a \neq b^p \implies \log_b(a) \neq p$.

- **Part (b):**

Using master's theorem: $S(n) = \theta(n^{\log_b(a)})$.

$$\begin{aligned} S(n) = T(n) - c.f(n) &\implies T(n) = S(n) + c.f(n) = S(n) + c.n^p \\ &= \theta(n^{\log_b(a)}) + \frac{b^p}{a - b^p}.n^p = \theta(n^{\log_b(a)}) - \frac{b^p}{b^p - a}.n^p \end{aligned}$$

- **Part (c):** This is just plug-and-play using Master's theorem Case 1:

1. $\theta(n^{\log_2(3)}) + 2n$
- ii. Undefined due to division by 0.
- iii. $\theta(n^{\log_3(5)}) - \frac{9}{4}n$.

3 Question 3

- **Part (a):**

Each matrix has $\frac{n^4}{4}$ elements. Allocating 21 of these uses $21 \times \frac{n^2}{4} \approx O(n^2)$. Then 7 recursive calls on input of size $\frac{n}{2}$. So the recurrence comes out as:

$$S(n) = S(n/2) + c.n^2; a = 7, b = 2, f(n) = cn^2.$$

Using master's theorem: $S(n) = O(n^{\log_2 7})$.

- **Part (b):**

The 7 calls to *MM-STANDARD-HYBRID* are made each of which call *MM-STRASSEN-HYBRID* 8 times on matrices of size $\frac{n}{2} \times \frac{n}{2}$. So the recurrence looks like:

$$S_h(n) = (7 \times 8)S_h(n/4) + cn^2 = 56.S_h(n/4) + cn^2.$$

Using the values for a and b from this, and using master's theorem give: $O(n^{\log_4 56})$.

- **Part (c):**

Despite the HYBRID method allocation less space per matrix, the number of recursions per level is way more than the basic method. As the recursion gets deeper, the HYBRID approach being bottom-level heavy, each of these recursion calls leads to more space being allocated in the HYBRID method.

4 Question 4

Remove the leading bit from each number. Multiply the remaining $\frac{n}{2}$ bit numbers in $T(\frac{n}{2})$. If one is 1 and the other is zero, we add a copy of the number starting with a zero shifted by $\frac{n}{2}$ (from $(\frac{n}{2} + 1) - 1$). If both start with 1, we add a copy of one number shifted by $\frac{n}{2}$ (from $(\frac{n}{2} + 1) - 1$), and then we add a copy of the other number without its leading digit bit shifted by $\frac{n}{2}$ (from $(\frac{n}{2} + 1) - 1$). this is $\Theta(n)$ work.

It works because we're adding back in the missing leading bits with the bit shifted part of one number, and then doing the last multiplication with the bit shifted entire other number. Concretely, both $x_L + x_R$ and $y_L + y_R$ are $n/2 + 1$ bits. Separating the first bit, assuming it to be 1 gives: $2^{n/2} + x'$ and $2^{n/2} + y'$.

$$(2^{n/2} + x')(2^{n/2} + y') = 2^{n/2} \times 2^{n/2} + 2^{n/2} \times y' + 2^{n/2} \times x' + x'y'.$$

Each of the parts is a shift and add apart from the last term which is an $n/2$ -bit -by- $n/2$ -bit multiplication. Thus, even in the worst case there

is an overhead of $O(n)$ along with the multiplication over input of size $n/2$.
Therefore, we can write: $T(n/2 + 1) + O(n)$.